# Resumen de implementación y pruebas

### Informe técnico · 15 de octubre de 2026

Proyecto i4a / ComNetAR. Solución de enlaces redundantes entre nodos ESP32, recuperación de rutas y trabajos de estabilidad, operación y diagnóstico asociados.

## Resultado principal

Se implementó la búsqueda coordinada de un vecino alternativo, la admisión de una única sesión por par de nodos y la recuperación de salida y retorno sin renumerar los clientes durante los cortes ensayados. La documentación final del 12 de octubre registra cuatro recuperaciones exitosas, dos por cada nodo periférico, con pérdidas durante la transición.

## Alcance de la revisión

La comparación funcional toma como base de integración CoNexDat/i4a_project 8df7996 y como versión integrada feat/redundant-link-discovery 81aa146. Se incorporan las notas de pruebas del 12 de octubre y las auditorías e instrumentación del ensayo prolongado del 15 de octubre. Estas últimas son trabajos posteriores y no deben atribuirse íntegramente al commit 81aa146.

## Criterio de evidencia

Este documento resume código, documentación y resultados registrados en el worktree three-node-soak-20261015. No se ejecutaron nuevamente las pruebas ni se operaron las placas al elaborar el informe. Las cantidades de casos corresponden a revisiones concretas: no se suman suites de fechas distintas como si fueran casos independientes.

## Límites principales

La recuperación ensayada no acredita ausencia de fallos prolongados, pérdida simultánea de enlaces ni un caudal garantizado. Persisten degradación con carga concurrente, CRC inválidos al recoger DMA e incertidumbres sobre los timeouts OTA observados en hardware.

# 1. Arquitectura y comportamiento implementado

### Modelo del sistema

Cada nodo lógico contiene cinco ESP: un central y cuatro periféricos orientados. El anillo SPI transporta coordinación interna y tráfico; las radios periféricas establecen enlaces Wi-Fi/TCP con otros nodos. El UUID identifica al nodo completo, no a una placa individual.

### Topología de redundancia

La prueba utiliza Root, Ramona y Grillo. Además de los enlaces iniciales hacia Root, se admite un lateral Ramona–Grillo. Al interrumpir un uplink, el nodo afectado puede llegar a Root a través del otro nodo. El éxito exige actualizar tanto la salida como la ruta de retorno hacia la subred del cliente.

### Descubrimiento controlado

discovery_manager, nuevo, arbitra desde el central un único turno de búsqueda. Propietario, sesión de arranque y secuencia evitan que solicitudes o liberaciones atrasadas afecten una búsqueda posterior. En configuración normal se espera en AP entre 600 y 700 segundos; el intento local dura hasta 60 segundos y la reserva central 90 segundos. El arranque del central tiene una cuarentena de 90 segundos.

### Control de modos

node_control concentra las transiciones AP/Station. Station verifica el turno antes del escaneo y antes de asociarse; su cancelación es cooperativa para no eliminar tareas que retengan mutex. El intento fallido retorna a AP y libera el turno. Un enlace ya admitido y una asociación AP provisional ocupada no se interrumpen para buscar redundancia.

### Reglas de direccionamiento

offer_policy valida la red lógica provided_network. Para una oferta redundante, su base y su longitud de prefijo deben ser mayores que las del bloque original del nodo. No se usa el /30 de DHCP para esa comparación. La provisión inicial tiene sus propias validaciones; restaurar el uplink acepta su asignación original. Una oferta rechazada no renumera al nodo.

# 2. Admisión, transporte y ciclo de vida

### neighbor_manager y peer_session: componentes nuevos

El registro central de vecinos permite un único enlace por UUID remoto. La sesión intercambia identidad, roles y orientaciones; reserva, compromete y activa el enlace antes de entregar datos o eventos al ruteo. El orden determinista de reservas resuelve intentos simultáneos; un enlace comprometido o activo conserva prioridad.

### Renovación y liberación

Los tokens y orientaciones identifican cada sesión: una liberación vieja no elimina una sesión posterior. Las copias SPI tienen revisión. Las reservas pendientes vencen a los 20 segundos; un activo sin renovación, a los 30; la renovación normal ocurre cada 5 segundos. La falta de confirmación impide admitir o mantener un enlace como si la coordinación hubiera sido exitosa.

### client, server y eventos

Se agregó framing de longitud para reconstruir mensajes fragmentados o agrupados por TCP. La identidad I4AN v1 precede al ruteo. Los eventos se entregan ordenados y una asociación rechazada no crea rutas. El servidor conserva un supervisor que reintenta fallos de socket, bind, listen o select cada 2 segundos hasta el cierre solicitado.

### device, station, access_point y routing_hooks

El cierre detiene tareas TCP y retira referencias de interfaz antes de destruirlas; los handlers ignoran eventos tardíos. La clasificación punto a punto exige una máscara válida y, en Station, IP obtenida y ausencia de cierre. Esto corrige el caso máscara=0/subred=0, que capturaba cualquier destino durante el escaneo y desviaba tráfico de tránsito hacia una interfaz sin conexión.

### Configuración de radio

config_portal, portal_settings y wifi_credentials incorporan configuración persistida; Station aplica el umbral RSSI por interfaz. antenna_orientation_mode agrega modos especiales. El filtro RSSI ayuda a seleccionar candidatos, pero no garantiza eliminar lóbulos traseros. Se desactiva modem-sleep para reducir espera de recepción, con mayor consumo eléctrico.

# 3. Recuperación y consistencia de rutas

### routing: salida, distancia y retorno

Se retiran rutas asociadas a la interfaz perdida y se invalida el gateway que la usaba. Se preservan las subredes del nodo, de los cinco módulos y de los clientes. Root tiene DTR 1 y cada enlace inter-nodo agrega un salto; elegir gateway por SPI no suma otro. Los candidatos se comparan por costo completo y se rechazan valores inválidos o que desbordan.

### Búsqueda de gateway

No se interpretan entradas vacías de hag_networks como rutas 0.0.0.0/0. Las solicitudes se reintentan cada 30 segundos y Root agenda su respuesta 10 segundos después del primer pedido. Pedidos adicionales no postergan ese plazo. Sin gateway se usa una interfaz de descarte con ERR_RTE para evitar el fallback de lwIP a SPI.

### Corrección de retorno tras perder un enlace no predeterminado

La recuperación HAG se inicia también cuando una radio provisionada con DTR mayor que 1 pierde un enlace que ya no era el gateway activo. Antes, Root podía retirar la ruta directa sin recibir el anuncio de retorno por el lateral. Root conserva su salida central. Esta política puede provocar convergencia adicional ante otras pérdidas.

### sync y shared_state: coordinación v2

El token lleva sesión y secuencia. El central reintenta la misma circulación cada 5 segundos; el periférico no repite un callback completado. Una cola pendiente vuelve a solicitar turno cada 3 segundos. Las colas de ruteo usan mutex y lotes separados. Las tablas compartidas incluyen versión, sesión, secuencia y propietario; una publicación fallida queda pendiente antes de consumir más eventos.

### Compatibilidad

Se conserva un único gateway activo y se reutiliza la búsqueda existente; no se agregó una lista priorizada de gateways de respaldo. STARTUP, SYNC y SHARED_STATE v2 requieren firmware compatible en las cinco placas de cada nodo. Las imágenes caben en las particiones existentes; no se requiere borrar NVS ni cambiar el layout.

# 4. OTA, estabilidad y observabilidad

### Mantenimiento y arranque

node_ota, ring_share, reset_manager y app_main coordinan mantenimiento y barrera de arranque. Se bloquean nuevas admisiones y turnos durante mantenimiento. Los callbacks encolan trabajo; las escrituras flash y ACK se realizan fuera del callback compartido. Se serializan las operaciones de las periféricas antes de la operación local del central.

### Pausa del transporte

La compuerta inferior bloquea IP local y reenviado, pero permite tráfico INTERNAL. ring_share permite OTA y RESET durante maintenance. El reenvío de broadcast ocurre antes de la entrega local. La auditoría del 15 de octubre verificó hashes y fuentes críticas de los bundles; esto acredita la presencia de los mecanismos, no la ausencia de bloqueos en ejecución.

### Memoria y SPI

ring_link_netif entrega el pbuf a lwIP o lo libera ante rechazo/interfaz caída, evitando retención bajo saturación. SPI agrega diagnóstico CRC al completar DMA y antes del procesamiento. CONFIG_SPI_RX_EXTRA_BUFFERS permite reservas adicionales sin cambiar los ocho descriptores DMA, la trama de 1600 bytes ni SPI a 8 MHz. El perfil extra8 es experimental y aumenta el uso estático de memoria.

### Diagnósticos

I4AROUTE registra eventos de sesión, cola, token, tabla y gateway mediante un worker UART. info_manager distingue peer_admitted y edad de muestras. Los contadores por tramo separan CRC, tamaño inválido, colas y fallos de transporte. Una muestra periódica no equivale a captura continua ni a prueba de entrega remota.

### Ensayo prolongado preparado el 15 de octubre

Se prepararon imágenes soak con INFO, core dump ELF por UART, estadísticas y muestreo I4ASOAK de heap/pilas. Se propuso una referencia de 12–24 h y pruebas de carga y reconexión separadas. No se encontró en estas fuentes un resultado final de esa corrida. Soak difiere del build local: TCP/IP 3072 frente a 4096, tiempos normales frente a rápidos, más instrumentación y cero buffers adicionales frente al buf8 local de Ramona. No son referencias equivalentes.

# 5. Pruebas de software documentadas

### Admisión y descubrimiento

La nota inicial de redundancia registra 34 pruebas Python de bundles/actualización, 10 nativas del árbitro, 10 de STARTUP/portal/persistencia RSSI/escaneo y la prueba nativa de ofertas aprobadas. La protección de vecinos incluye intercalaciones recíprocas, expiración, renovación, sesiones antiguas, fragmentación TCP y orden de eventos.

### Coordinación y recuperación

La revisión v2 registra 37 pruebas Python y simulaciones del código real sync/shared_state: pérdidas por salto, entregas parciales, duplicados y tablas fallidas. Se verificó reintento del listener tras un bind fallido. Las pruebas de recuperación inicialmente bloqueadas por Windows se ejecutaron después en RPi: 14 casos y hook sin salida aprobados. Tras la corrección de retorno se registran 15 casos de recuperación y 37 Python aprobados; la regresión específica fallaba antes del cambio.

### Punto a punto, memoria y buffers

Cuatro casos nativos verifican escaneo sin IP, máscara cero, transporte válido y estados ausentes/detención. El callback netif se probó con ambos modos de reporte de errores: 10.000 entregas por modo, 7.500 rechazos por saturación y escenarios de interfaz inválida/caída. SPI ejecutó tres escenarios con extra0 y extra8: seis ejecuciones aprobadas sobre tamaño de trama, retención de buffers e integridad CRC.

### Auditoría OTA del 15 de octubre

run_native_ota.py: 15 casos aprobados, más compatibilidad C/Python y pruebas de corrupción/truncamiento. run_native_broadcast.py: aprobado el orden de reenvío bajo presión. En neighbors aprobaron SPI, lowlevel, ring, registro y manager; la batería se detuvo al enlazar session_test.c por falta del mock os_diag. No se considera aprobada la batería completa.

### Qué prueban y qué no

Las pruebas nativas ejecutan funciones C reales con proveedores simulados. Los builds ESP-IDF 5.4.1 acreditan compilación y ajuste en particiones. No reproducen cinco schedulers, DMA eléctrico, RF, flash y tráfico real simultáneo. Los resultados pertenecen a las imágenes y revisiones indicadas en cada nota.

# 6. Pruebas físicas de recuperación

## Fallas previas y diagnóstico

Los ensayos previos mostraron gateway local accesible y destinos remotos sin respuesta durante el corte. Las capturas correlacionadas llegaron a mostrar solicitud y respuesta en RPi mientras el cliente agotaba el tiempo: la salida funcionaba, pero fallaba el retorno dentro de la malla. La selección de Sur en la tabla local por sí sola no validaba la conmutación.

## Instalación verificada

La corrección de retorno se instaló en las cinco placas de Ramona, Grillo y Root. Los reportes registraron hash esperado, ready=1, boot_paused=0, pending_verify=0 y recovery_pending=0. Se verificaron referencias gateway/RPi/internet y rutas del PC antes de cortar.

## Resultados finales del 12 de octubre

Cuatro cortes de 60 segundos: dos sobre Este de Ramona y dos sobre Oeste de Grillo. Las respuestas remotas volvieron antes del vencimiento de cada corte. Las repeticiones incluyeron gateway alternativo ya activo. Las IP se conservaron durante cada ensayo: Ramona 10.116.0.2 y Grillo 10.148.0.2.

| Corte | Gateway | RPi / Internet | Intervalo observado |
|---|---|---|---|
| Ramona 1 | 143/143 | 134/143 ambos | ~11,3 s |
| Ramona 2 | 137/137 | 130/137 ambos | ~9,9 s |
| Grillo 1 | 148/148 | 139/148 ambos | ~11,2–11,3 s |
| Grillo 2 | 133/133 | 126/133 ambos | ~9,8 s |

## Interpretación y límites

Los cocientes son respuestas ICMP sobre intentos del ensayo completo. Los tiempos van del primer timeout a la respuesta posterior; no son tiempos exactos desde el corte físico. Se usaron pings del PC y consolas centrales, sin nueva captura RPi simultánea en estos cuatro ensayos. Se confirmó por telemetría la restauración de los enlaces originales tras ensayos previos; falta la confirmación posterior al cuarto. No se demuestra retorno automático del gateway predeterminado ni tolerancia a dos pérdidas simultáneas.

# 7. Carga simultánea y comparación de buffers

### Caudal y pausas TCP

Con cuatro conexiones por nodo, timeout de 10 s y fases de 60 s, Grillo solo alcanzó 1,907 Mbit/s; con ambos cargados, Ramona 2,107 y Grillo 0,665 Mbit/s, sin errores de aplicación. La degradación de Grillo fue aproximadamente 65% en ese ensayo. El PCAP mostró repetidos exactos de Grillo de 3,92% a 6,34% y pausa máxima de 1,074 a 6,281 s. Son métricas del analizador, no un porcentaje de pérdida radio.

### Repetición con contadores

Otro ensayo registró Grillo solo 1,468 Mbit/s y, con ambos, Ramona 1,475 / Grillo 0,690 Mbit/s. Hubo CRC descartados en los centrales, pero su distribución no explica por sí sola la degradación. No se localizó el salto de pérdida entre SPI, radio, colas, servidor y host cliente.

### Comparación extra0 / extra8 en Ramona

Con ambos nodos cargados, Ramona obtuvo 1,382 frente a 1,381 Mbit/s. La espera acumulada registrada para adquirir buffers pasó de unos 2,86 s en central y 2,93 s en Este a cero incremento en B; el contador solo suma adquisiciones de al menos 100 microsegundos. Los CRC del central fueron 29 frente a 21, todos ya inválidos al recoger DMA, sin cambios posteriores detectados.

### Conclusión acotada del experimento

Las reservas eliminaron las esperas registradas en la carga B, pero no demostraron aumento de velocidad ni eliminación de CRC. Grillo cambió de asignación/recorrido y transcurrieron horas entre A y B: no es una comparación causal controlada. Se midieron 12.896 bytes estáticos adicionales; no se midió heap libre real durante esa comparación. La variante buf8 quedó experimental.

### Pendientes prioritarios

Repetir A/B con topología, configuración y carga fijas; capturar periféricas y centrales por tramo; ejecutar soak prolongado con fuentes/configuración identificadas; correlacionar BEGIN, DATA, ABORT, colas y retorno del broadcast para localizar los timeouts OTA. La auditoría identifica posibles bloqueos de despacho, pero no demuestra su causa ni justifica declarar resuelto el problema.

# 8. Fuentes y trazabilidad

### Referencias de implementación

[1] docs/comnetar-integration.md: origen comnetar y base 8df7996. [2] docs/redundant-link-discovery.md: arbitraje, ofertas, tiempos y validación inicial. [3] docs/neighbor-link-protection.md: UUID, reservas y transporte. [4] docs/gateway-route-recovery.md: gateway, DTR, hooks y pbuf.

### Correcciones y pruebas funcionales

[5] docs/correcciones-coordinacion-v2.md: token/tablas v2, servidor y ejecución en RPi. [6] docs/correccion-transporte-vacio-20261012.md: máscara cero, fallas observadas y regresión de retorno. [7] docs/instalacion-retorno-20261012.md: instalación y cuatro cortes finales; sus actualizaciones finales reemplazan pendientes históricos anteriores.

### Carga y transporte

[8] docs/carga-simultanea-20261012.md: throughput y análisis PCAP. [9] docs/manejo-buffers-spi-20261012.md: diseño, pruebas y comparación extra0/extra8. [10] docs/evidencia/20261012/README.md y manifest.json: resultados preservados, procedencia, tamaños y SHA-256.

### Auditoría y ensayo prolongado

[11] docs/three-node-soak.md: imágenes, captura y protocolo de ensayo. [12] docs/ota-pause-audit-20261015.md: hashes, mecanismos OTA y ejecución parcial de neighbors. [13] docs/local-versus-soak-20261015.md: diferencias efectivas de fuentes y configuración.

### Ubicación de la evidencia

Las referencias anteriores se consultaron en el worktree three-node-soak-20261015. Parte de los JSON y CSV del 12 de octubre se preserva en docs/evidencia/20261012/. Las notas también apuntan a build/firmware/ para logs originales; PCAP completos y UART cruda permanecen locales y no todos los ensayos tienen capturas simultáneas. Un reporte installed histórico no acredita el firmware activo hoy.

### Reproducibilidad del informe

Este PDF y su versión Markdown se generan con docs/generar_resumen_pdf.py. Para revisar las modificaciones de producto: git diff 8df7996 81aa146 -- components app tests. Para repetir las pruebas, usar los runners y requisitos de cada documento, conservando commit, sdkconfig, imagen, ELF, hashes, topología y marcas temporales del ensayo.